main.py

```python
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)
    order = []

    while queue:
        vertex = queue.popleft()
        order.append(vertex)

        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

    return order


def dfs(graph, start, visited=None, order=None):
    if visited is None:
```

```python
    if visited is None:
        visited = set()

    if order is None:
        order = []

    visited.add(start)
    order.append(start)

    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited, order)

    return order


# Representing graph as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
```

```python
    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited, order)

    return order


# Representing graph as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D', 'E'],
    'C': ['A', 'F'],
    'D': ['B'],
    'E': ['B', 'F'],
    'F': ['C', 'E']
}

print("BFS Traversal starting from 'A':", bfs(graph, 'A'))
print("DFS Traversal starting from 'A':", dfs(graph, 'A'))
print("Name: Moulya M P")
print("Register Number: 24BECS167")
```

Run

Output

```
BFS Traversal starting from 'A': ['A', 'B', 'C', 'D', 'E', 'F']
DFS Traversal starting from 'A': ['A', 'B', 'D', 'E', 'F', 'C']
Name: Moulya M P
Register Number: 24BECS167

=== Code Execution Successful ===
```